SEN 306 – Software Construction

Design and Implementation of a Modular Python Application Using Stack, Queue, and Singly Linked List

Name: Oluwadunsin Omoniyi

Reg No: 2024/B/SENG/0297

Date: 01/10/2026

---

1. Introduction

Software construction is the process of designing, developing, testing, and maintaining software systems using accepted programming principles and best practices. One of the key goals of software construction is to create software that is reliable, reusable, maintainable, and easy to understand.

This project involves the development of a modular Python application that demonstrates the implementation and interaction of three fundamental data structures: Stack, Queue, and Singly Linked List. These data structures are commonly used in software development because they provide efficient ways of storing, organizing, and manipulating data.

The application simulates a simple student support management system. Student records are maintained using a linked list, support requests are managed using a queue, and processed requests are stored using a stack. Through this approach, the project demonstrates how different data structures can work together within a single software solution.

The project was developed using software construction principles such as modularity, high cohesion, low coupling, readability, and code reusability. Each data structure was implemented in a separate Python module and integrated through a main program.

---

2. Project Objectives

The objectives of this project are:

- To implement a Stack data structure using Python classes and methods.
- To implement a Queue data structure using Python classes and methods.
- To implement a Singly Linked List data structure using Python classes and methods.
- To demonstrate the interaction between the three data structures in a real-life application.
- To apply software construction principles in the design and implementation of the system.
- To develop a modular and maintainable software solution.

By achieving these objectives, the project provides practical experience in both data structures and software engineering practices.

---

3. Project Structure

The application was organized into separate modules to promote modularity and maintainability.

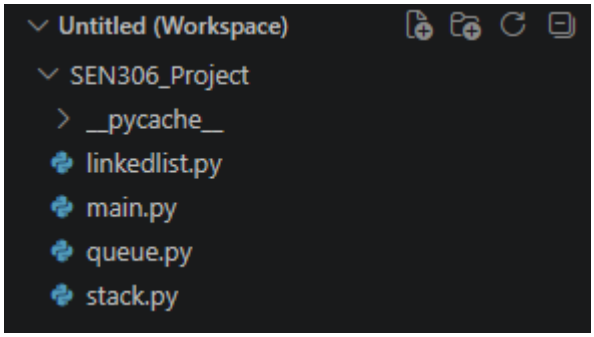
The project consists of the following files:

- **stack.py**
- **queue.py**
- **linkedlist.py**
- **main.py**

Each module contains the code related to a specific data structure, while the main file coordinates the overall operation of the application.

This approach makes the system easier to understand because each file has a clearly defined responsibility. It also improves code reuse because the modules can be imported into other projects without modification.

Figure 1: Project Folder Structure



---

4. Implementation of Data Structures

4.1 Singly Linked List

The Singly Linked List was used to store student records.

A linked list consists of nodes where each node stores data and a reference to the next node in the sequence. Unlike arrays, linked lists can grow dynamically during program execution.

In this project, the linked list stores the names of registered students. The following students were added:

- Andre
- Tolu
- Aisha

The linked list allows records to be inserted and traversed efficiently. During execution, the contents of the linked list were displayed to confirm that the student records had been stored successfully.

The use of a linked list demonstrates dynamic data storage and efficient record management.

4.2 Queue

The Queue was used to manage support requests submitted by students.

A queue follows the **First-In-First-Out (FIFO)** principle. This means that the first item added to the queue is the first item removed.

The following support requests were added:

- Andre Complaint
- Tolu Complaint
- Aisha Complaint

When the requests were processed, Andre's complaint was handled first because it was the earliest request submitted.

Queues are commonly used in real-life situations such as customer service systems, printer scheduling, task management systems, and operating systems. Their use in this project demonstrates how requests can be handled fairly and systematically.

4.3 Stack

The Stack was used to store processed requests.

A stack follows the **Last-In-First-Out (LIFO)** principle. This means that the most recently added item is the first item removed.

After Andre's complaint was processed from the queue, it was pushed onto the stack. The stack then enabled an undo operation by retrieving the most recently processed request and returning it to the queue.

Stacks are widely used in software applications for features such as undo operations, browser history, expression evaluation, and function call management.

The implementation in this project demonstrates a practical example of how stacks can support recovery and reversal of actions.

---

5. Program Execution and Results

The application was executed through the `main.py` file.

During execution, the following sequence of events occurred:

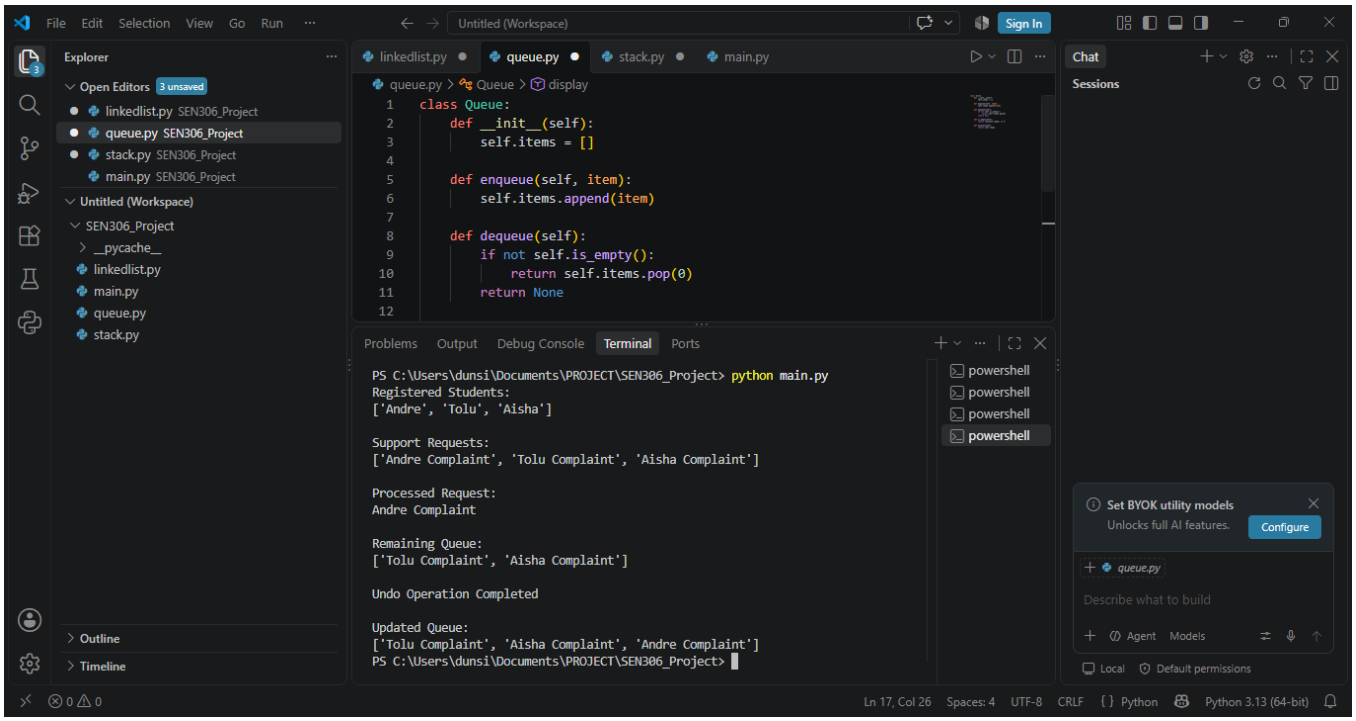
1. Student records were added to the linked list.
2. Support requests were added to the queue.
3. The list of registered students was displayed.
4. The queue of support requests was displayed.
5. The first request in the queue was processed.
6. The processed request was stored in the stack.

7. An undo operation was performed.
8. The request was restored to the queue.
9. The updated queue was displayed.

The output produced by the program confirmed that all operations were executed successfully.

The successful execution demonstrates that the data structures were implemented correctly and that they interacted effectively within the application.

Figure 2: Program Output



---

6. Application of Software Construction Principles

Several software construction principles were applied throughout the development of this project.

6.1 Modularity

The system was divided into separate modules, each responsible for a specific task. This improves maintainability and allows individual components to be modified without affecting the entire application.

6.2 High Cohesion

Each class and module was designed to perform a single well-defined responsibility.

For example:

- The Stack class manages stack operations only.
- The Queue class manages queue operations only.
- The Linked List class manages linked list operations only.

This results in a cleaner and more organized system.

6.3 Low Coupling

The modules have minimal dependency on one another.

The main program interacts with each module through clearly defined methods. As a result, changes made to one module are unlikely to affect the others.

6.4 Reusability

Because each data structure was implemented independently, the modules can be reused in future projects without significant modification.

This reduces development time and promotes efficient software construction.

6.5 Readability and Maintainability

The source code was written using meaningful names, proper indentation, and a consistent coding style.

Examples of meaningful method names include:

- push()
- pop()
- enqueue()
- dequeue()
- insert()

These naming conventions improve understanding and simplify future maintenance.

6.6 Documentation

Comments and structured code organization were used to improve clarity and make the program easier for other developers to understand.

Proper documentation is an important aspect of software construction because it supports maintenance and future enhancements.

---

7. Conclusion

This project successfully implemented three important data structures—Stack, Queue, and Singly Linked List—using Python. The application demonstrated how these data structures can be integrated into a practical student support management system to manage records, process requests, and support undo functionality.

The project also provided an opportunity to apply important software construction principles including modularity, high cohesion, low coupling, readability, maintainability, and reusability. Through the successful execution of the application, all project objectives were achieved.

Overall, the project strengthened understanding of both data structures and software construction practices while demonstrating the importance of well-structured and maintainable code in software development.